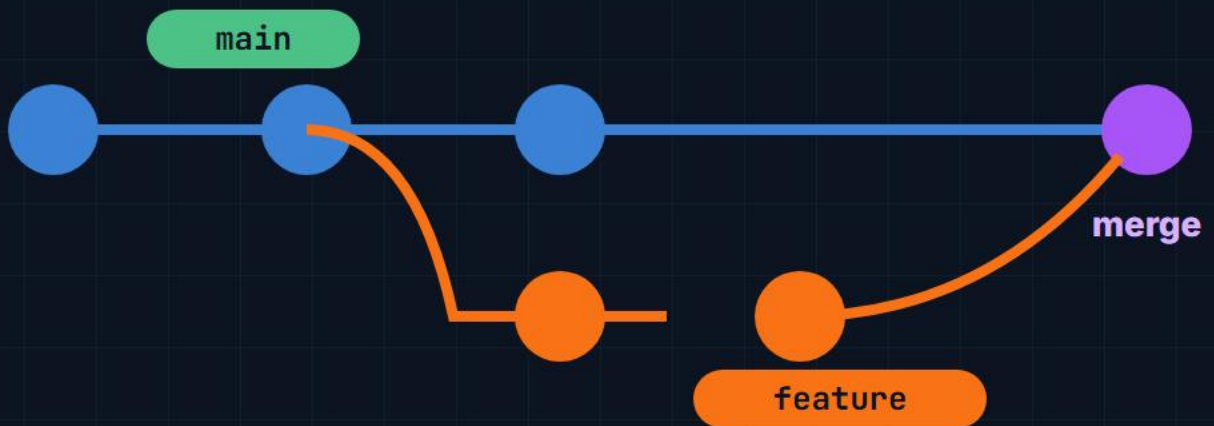


DEVOPS PROJECT · EXPLAINED VISUALLY

Git & GitHub: 98 lessons, every command tested

From your first commit to a release deployed on Kubernetes.



98

hands-on lessons

18

troubleshooting labs

1,308

code blocks tested

Understand → Visualize → Run → Break → Fix

every output in the lessons is a real output

THE PAIN

Tutorials teach commands. Work brings errors.

What actually happens in a team repository:



Merge conflict

```
<<<<<<< HEAD  
latte 3.30  
>>>>>> feature
```



"I lost my work"

```
git reset --hard HEAD~3  
three commits gone...  
(the reflog has them)
```



Push rejected

```
! [rejected] main → main  
(fetch first)
```



Secret committed

```
git rm does not remove  
it from history
```

THE METHOD

Every lesson: **break it on purpose**

The same ten steps for all 98 concepts.

1 · Understand

2 · Visualize

3 · Execute

4 · Observe

5 · Break

6 · Troubleshoot

7 · Fix

8 · Practice

9 · Challenge

10 · Real world

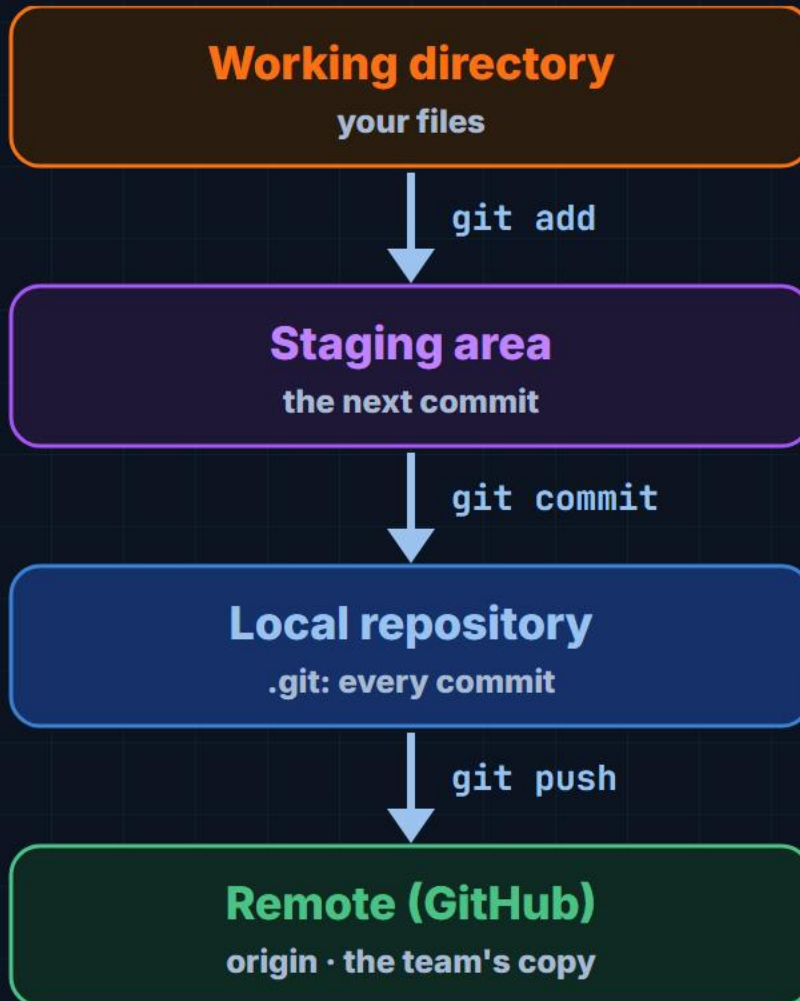
each lab starts from a known history:

same commit IDs on your screen as in the lesson

CONCEPT 1

Where your changes live

Four places. Most confusion comes from not knowing which one you are in.



CONCEPT 2

git reset: **three modes**

All three move the branch. The difference is what happens to your work.

	branch	staged	your files
--soft	moves	kept	kept
--mixed	moves	reset	kept
--hard	moves	reset	reset!

Committed work survives all three: git reflog

Shared branch? Use git revert instead: it adds, never rewrites.

CONCEPT 3

"I lost my commits" → the reflog

Real output from lesson 33, after an accidental reset:

```
$ git reflog
```

```
6833580 HEAD@{0}: reset: moving to HEAD~3
```

```
ecff18a HEAD@{1}: commit: Price mocha
```

```
2c389c0 HEAD@{2}: commit: Add mocha
```

```
269869e HEAD@{3}: commit: Price green tea
```

```
$ git reset --hard 'HEAD@{1}'
```

```
HEAD is now at ecff18a Price mocha
```

recoverable

committed work, deleted
branches, dropped stashes,
staged files (fsck)

gone

edits never staged or
committed, then reset
--hard or restored

18 TROUBLESHOOTING LABS

Errors you will meet **at work**

Each one reproduced, investigated, fixed and prevented.

CONFLICT (content): Merge conflict in prices.txt

! [rejected] main → main (non-fast-forward)

git@github.com: Permission denied (publickey).

GH006: Protected branch update failed

fatal: refusing to merge unrelated histories

GH001: Large files detected

THE CAPSTONE, LIVE ON GITHUB

A broken repo → Kubernetes

History cleaned, main protected, PRs through checks, v1.0.0 released and deployed.



feature branch



pull request



review



required checks



merge to protected main



tag v1.0.0



GitHub Actions



image → GHCR



Helm release



Kubernetes (kind)

completed success

job release: success

job deploy: success

WHAT IS INSIDE

A whole course, **free**

Lessons, labs, projects, videos and study material.

98

lessons with labs

20

module assessments

18

troubleshooting labs

6

projects + capstone + exam

23

videos, full and silent

125

page study guide PDF

HOW I KNOW IT WORKS

The lessons are **tests**

Every Bash block runs in CI; the outputs you read are the real ones.

```
<!-- test: fail; contains=CONFLICT →
```

```
```bash
```

```
git merge feature-tea
```

```
```
```

✓ 1,308 blocks · latest Git · every push

a sandbox per run

temporary HOME, own
~/.ssh and kubeconfig,
no prompts, no pager

real GitHub too

153 blocks ran against
GitHub: PRs, reviews,
protection, releases, CI

TRY IT

Clone it, **break it**, fix it

Lesson 01 needs only Git and a terminal.



The course



All my projects

Which Git error cost you
the most time? 💬